ML Study Guide - Part 1: Learn Every Topic

This guide assumes you know nothing. Read it like a patient tutor is sitting next to you, explaining everything from scratch. Every concept is explained in plain words FIRST, then we slowly build to the formula, then we use the formula on an example.

Chapter 1: What Is Machine Learning?

Start here: the simplest possible explanation.

Imagine you show a 3-year-old child 100 photos. Some are cats, some are dogs. Every time you show a photo, you say "cat" or "dog." You never explain any rules. You never say "cats have pointy ears." You never say "dogs have wet noses." You just show photo after photo with the label.

After 100 photos, something amazing happens. You show the child a *brand-new* photo they have never seen before, and they say "cat!" How? They figured out the patterns on their own. Nobody programmed rules into them. They *learned* from examples.

That is machine learning. Instead of a human writing rules ("IF ears are pointy AND body is small THEN cat"), we give a computer lots of examples (data + correct answers), and the computer discovers the rules by itself. The more examples it sees, the better it gets.

In traditional programming: Human writes rules + data goes in = answers come out.
In machine learning: Data + answers go in = rules come out.

Tom Mitchell's Formal Definition

A program is said to **learn** from experience **E**, with respect to some task **T**, and some performance measure **P**, if its performance on T (as measured by P) improves with experience E.

In simple words: **T** = "what do you want the computer to do?" **E** = "how does it practice?" **P** = "how do you grade it?" If the grade gets better as it practices more, the computer is learning.

4 Real-World Scenarios:

Scenario	T (What to do)	E (How it practices)	P (How to grade it)	Type
Self-driving car	Navigate roads safely	It drives around. Good moves get rewards, bad moves get penalties.	Average distance driven without mistakes	**Reinforcement learning** -- trial and error with feedback
Spam filter	Label new emails as spam or not-spam	Thousands of emails already labeled by humans	% of emails correctly classified	**Supervised classification** -- learns from labeled examples, output is a category
DNA clustering	Group similar gene sequences together	Unlabeled gene sequences (nobody said which group each belongs to)	How "pure" the resulting groups are	**Unsupervised** -- no labels, algorithm finds hidden groups on its own
Rainfall prediction	Predict how many mm of rain tomorrow	Years of historical weather data + actual rainfall amounts	How close predictions are to reality	**Supervised regression** -- output is a continuous number, not a category

Three Types of Learning (Paradigms)

1. Supervised Learning: You study for an exam with an answer key. For every practice question, you can check the correct answer. You compare your answer to the correct one and learn from mistakes. The data has *labels* (correct answers).

2. Unsupervised Learning: Someone dumps 500 socks on your bed and says "organize these." Nobody tells you how. You might sort by color, size, or pattern. You discover the groupings yourself. The data has *no labels*.

3. Reinforcement Learning: Training a puppy. It sits when asked -- you give a treat (reward). It chews your shoe -- you say "no!" (penalty). Over time, the puppy learns which behaviors lead to treats. The algorithm takes actions in an environment and gets rewards/penalties.

Classification vs Regression

The simplest test ever: Look at the output. **Can you list all possible answers?**

YES, I can list them: "cat" or "dog" or "bird." "spam" or "not spam." "A" or "B" or "C" or "D." The output is a *category* from a fixed list. This is **Classification**.

NO, the output is any number on a continuous scale: House price could be 4,50,000 or 4,50,001 or 7,23,456.78 -- infinite possibilities. This is **Regression**.

Quick practice:
"Will it rain tomorrow?" (yes/no) = classification (2 categories).
"How many mm of rain?" = regression (any number).
"What letter grade?" (A/B/C/D/F) = classification (5 categories).
"What exact percentage score?" = regression (any number from 0 to 100).

Parameters vs Hyperparameters

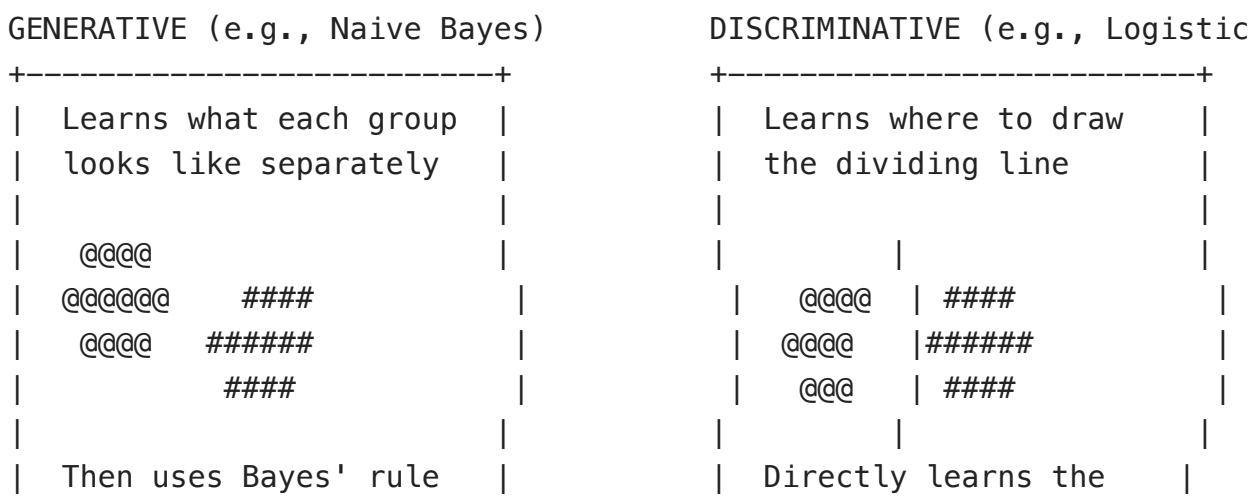
> Think of a student preparing for exams:
>
> **Parameters** = the student's actual knowledge. The things that *change as the student studies*. In ML, these are the weights and coefficients inside the model. You don't set these -- the algorithm figures them out from data during training. Example: the slope and intercept of a regression line.
>
> **Hyperparameters** = the study plan decisions made *before* studying begins. Which textbook to use? How many hours per day? These don't change while studying -- they are fixed choices that control HOW the learning happens. In ML: learning rate (how big each step is), k in kNN (how many neighbors to look at), max tree depth (how complex the tree can get). You choose these, run training, check results, and maybe adjust.

Lazy vs Eager Learners

> **Eager learner (Decision Tree, Linear Regression):** Studies hard *before* the exam. During training, it builds a complete model (a formula, a tree structure). When a test question comes, it answers instantly using its pre-built model. Slow to train, fast to predict.
>
> **Lazy learner (kNN):** Does not study at all. It just memorizes all the training data. When a test question comes, it looks at the question, searches its memory for the most similar past examples, and copies their answers. Zero training time, but slow to predict (must search through everything each time).

Generative vs Discriminative Models

> **Imagine two security guards at a party with VIPs and regular guests.**
>
> **Generative guard (e.g., Naive Bayes):** This guard separately studies what VIPs look like (tall, wear suits, expensive watches) and what regular guests look like (casual clothes, sneakers). For each group, the guard builds a detailed mental picture. When a new person arrives, the guard compares them to both mental pictures and asks: "Does this person look more like my VIP picture or my regular guest picture?" Technically, it learns P(features | class) -- "what do members of this class look like?"
>
> **Discriminative guard (e.g., Logistic Regression):** This guard doesn't care what each group looks like internally. It just learns the *boundary*. "People with gold badges = VIP. No gold badge = regular guest." It doesn't know anything about suits or sneakers -- it only knows the dividing line. It learns P(class | features) -- "given what I see, which class is it?"
>
> **Why does this matter?** Discriminative models are usually more accurate for classification because they focus only on the boundary. Generative models can do more (like generate new samples: "draw me a typical VIP") and handle missing data better.



```
|  to compare and decide   |          |  boundary, nothing else |
+--------------------------+          +-------------------------+
```

Chapter 2: Data Preprocessing

> **Before cooking, you wash your vegetables.** Real-world data is messy. It has blank cells, typos, duplicate rows, crazy outliers, and inconsistent formats. If you feed this mess into an ML algorithm, the algorithm will learn the mess, not the real patterns. Preprocessing = cleaning and preparing data so algorithms can work properly.

Spot the Problems (7 Issues in 6 Rows)

#	Name	Age	Email	Salary	Date	City
1	Ravi	25	ravi@mail.com	50000	2024-01-15	Mumbai
2	Priya		priya@mail.com	55000	15/01/2024	mumbai
3	Ravi	25	ravi@mail.com	50000	2024-01-15	Mumbai
4	Amit	30	amit@	900000	2024-02-10	Delhi
5	Sneha	28	sneha@mail.com	60000	2024-03-05	Pune
6	Kiran	22	kiran@mail.com	45000	2024-03-05	Pune

7 Issues:
1. **Missing value** (Row 2, Age is blank): The algorithm can't work with empty cells. Fix: fill with the average age of other rows, or drop the row.
2. **Duplicate** (Rows 1 & 3 are identical): Ravi gets counted twice, biasing the model. Fix: delete one copy.
3. **Outlier** (Row 4, salary 9,00,000 when others are 45K-60K): One extreme value can drag the entire model off course. Fix: investigate, cap, or remove.
4. **Inconsistent dates** (Row 2: DD/MM/YYYY vs others: YYYY-MM-DD): The algorithm might read "15" as a month and get confused. Fix: pick one format for all rows.
5. **Invalid data** (Row 4, email "amit@" has no domain): Not a real email. Fix: flag it or treat as missing.
6. **Inconsistent naming** ("Mumbai" vs "mumbai"): Computer sees these as two different cities. Fix: standardize all text to the same case.
7. **Redundant features** (If Name and Email always identify the same person, you don't need both): Extra columns add noise. Fix: drop the redundant one.

Feature Scaling: Why Your Algorithm Needs It

Let me explain the PROBLEM first, before any formula.

Suppose you are predicting house prices using two features: "Number of Bedrooms" (values: 1, 2, 3, 4, 5) and "Area in Square Feet" (values: 500, 1000, 2000, 3500, 5000).

Notice the scales are wildly different. Bedrooms go from 1 to 5 (range of 4). Area goes from 500 to 5000 (range of 4500). Area has numbers that are about 1000 times bigger than Bedrooms.

Why is this a problem? Many ML algorithms (linear regression with gradient descent, kNN, etc.) use all features together. When one feature has numbers in the thousands and another has numbers in single digits, the big-number feature *dominates everything*. The algorithm acts as if the small-number feature barely exists.

Think of it this way: In kNN, we measure "distance" between two houses. House A: 2 bedrooms, 1000 sqft. House B: 4 bedrooms, 1010 sqft. The distance between them is roughly sqrt((2-4)^2 + (1000-1010)^2) = sqrt(4 + 100) = sqrt(104). The 10 sqft difference contributes 100 to the distance, while the 2-bedroom difference contributes only 4. So kNN thinks these houses are similar, completely ignoring the huge bedroom difference! The area feature "drowns out" the bedroom feature.

The fix: Make all features live on the same scale. That way, no single feature dominates. There are two common ways to do this.

Method 1: Z-Score Normalization (Standardization)

The idea in plain words: For each value, ask: "How far is this from the average, measured in units of 'typical spread'?"

If your test score is 80 and the class average is 70, you are 10 points above average. But is 10 points a lot? It depends on how spread out the class is. If everyone scored between 68 and 72, then 10 points above average is exceptional. If everyone scored between 30 and 100, then 10 points above average is nothing special.

Z-score captures exactly this: how far from average, relative to the spread (standard deviation).

The formula: z = (x - mean) / standard_deviation

What each piece does:
(x - mean): "How far is this value from the average?" This CENTERS the data. Values above average become positive, values below become negative, and the average itself becomes zero.

/ standard_deviation: "Divide by the typical spread." The standard deviation tells you how spread out the values typically are. Dividing by it re-scales everything so that one unit of distance now means "one typical spread." If your data was spread across thousands, dividing by the SD (which is also in the thousands) shrinks everything down. If your data was in single digits, dividing by a small SD keeps things proportional.

After z-scoring: Every feature has mean = 0 and standard deviation = 1. A z-score of +2 means "this value is 2 standard deviations above average." A z-score of -1 means "1 standard deviation below average." Now ALL features are measured in the same units: "standard deviations from the mean."

Worked Example (every step shown):
Suppose feature values = [10, 20, 30].

Step 1: Calculate the mean (average).
Mean = (10 + 20 + 30) / 3 = 60 / 3 = **20**.

Step 2: Calculate the standard deviation (SD).
First, find how far each value is from the mean:
10 - 20 = -10. Squared: 100.
20 - 20 = 0. Squared: 0.
30 - 20 = +10. Squared: 100.
Average of squared differences: (100 + 0 + 100) / 3 = 66.67.
Square root: SD = sqrt(66.67) = **8.16**.

Step 3: Apply z = (x - mean) / SD to each value.
z(10) = (10 - 20) / 8.16 = -10 / 8.16 = **-1.22** ("1.22 SDs below average")
z(20) = (20 - 20) / 8.16 = 0 / 8.16 = **0.00** ("exactly average")
z(30) = (30 - 20) / 8.16 = 10 / 8.16 = **+1.22** ("1.22 SDs above average")

The original values [10, 20, 30] became [-1.22, 0, +1.22]. Centered at zero, spread proportionally.

Method 2: Min-Max Scaling

The idea in plain words: Squish all values into the range [0, 1]. The smallest value in your data becomes 0. The largest becomes 1. Everything else falls proportionally in between.

Think of it as asking: "What percentage of the way from the minimum to the maximum is this value?"

The formula: x' = (x - min) / (max - min)

What each piece does:
(x - min): "How far is this value above the smallest value?" The smallest value gives 0. This shifts the bottom of the range to zero.

(max - min): "How wide is the total range from smallest to largest?" This is the denominator that scales everything proportionally.

Together: You get a fraction. What fraction of the total range is this value above the minimum? 0% (the minimum itself) to 100% (the maximum itself).

Worked Example: Values = [10, 20, 30]. Min = 10. Max = 30. Range = 30 - 10 = 20.

x'(10) = (10 - 10) / 20 = 0 / 20 = **0.0** (the minimum maps to 0)
x'(20) = (20 - 10) / 20 = 10 / 20 = **0.5** (the middle maps to 0.5)
x'(30) = (30 - 10) / 20 = 20 / 20 = **1.0** (the maximum maps to 1)

Now values are [0.0, 0.5, 1.0]. Neatly within [0, 1].

Z-Score vs Min-Max: When to Use Which?

Use Z-score when your data might have outliers. Why? Because Z-score uses the mean and standard deviation, which are calculated from ALL data points. One extreme outlier will affect them a bit, but not catastrophically. The outlier simply gets a large z-score (like +5 or -8), while normal values stay in the -2 to +2 range.

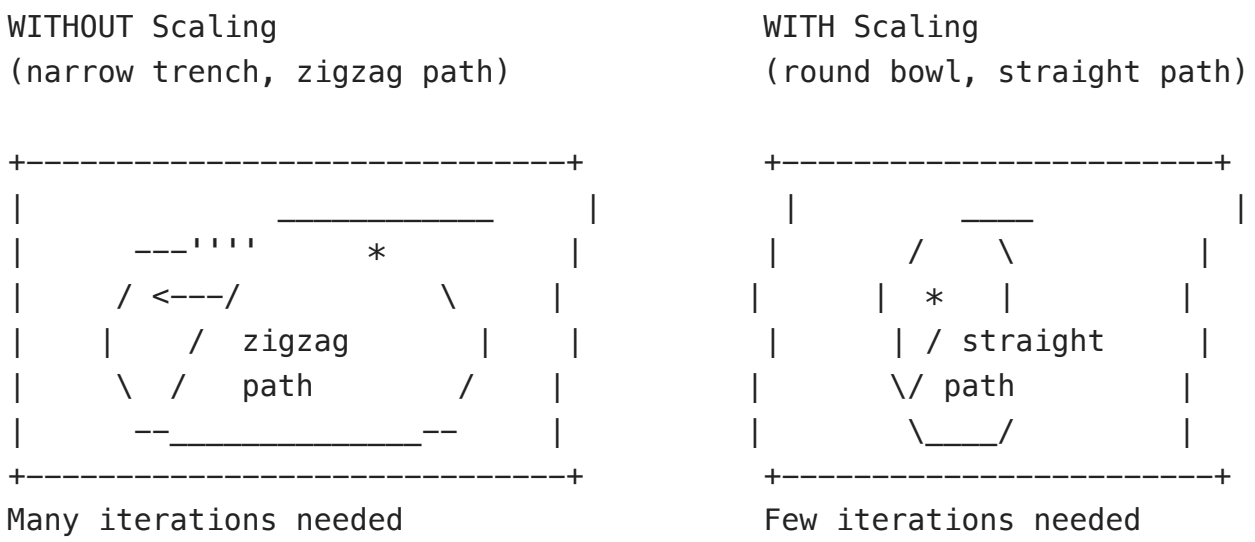
Avoid Min-Max when you have outliers. Why? Because Min-Max uses the minimum and maximum values. If most salaries are 40K-60K but one person earns 9,00,000, then min=40K, max=900K, range=860K. Now a salary of 50K maps to (50K - 40K) / 860K = 0.012. A salary of 60K maps to 0.023. All the normal salaries are squished into a tiny sliver between 0 and 0.03, while the outlier sits alone at 1.0. You lose all ability to tell normal salaries apart!

Use Min-Max when: You are confident there are no extreme outliers AND you need values strictly between 0 and 1 (e.g., feeding data into a neural network that expects inputs in [0,1]).

Scaling and Gradient Descent (The Zigzag Problem)

Why does scaling help gradient descent? Imagine gradient descent as a runner trying to reach the lowest point of a valley. If features are on very different scales, the cost function's shape is like a long, narrow trench (stretched along the big-number feature's axis). The runner can only feel the slope directly under their feet. In a narrow trench, the slope always points toward the walls (the narrow direction), not toward the bottom. So the runner bounces back and forth between the walls (zigzags) instead of heading straight toward the goal. This is painfully slow.

After scaling, the trench becomes a round bowl. The slope always points toward the bottom. The runner walks straight there in far fewer steps.



True/False Quick Check:

1. "Decision trees need feature scaling." FALSE. Trees split by asking "is x > 5?" They never compute distances between features. Scaling doesn't change the answer to threshold questions.

2. "kNN needs feature scaling." TRUE. kNN uses distance to find nearest neighbors. Without scaling, the feature with the biggest numbers dominates the distance calculation, and other features are essentially ignored.

3. "Min-Max is better than Z-score when there are outliers." FALSE. It's the opposite! Outliers destroy Min-Max by stretching the range. Z-score is more robust.

4. "Gradient descent converges faster with scaling." TRUE. Scaling turns the elongated cost contours into circular ones, so the gradient points directly toward the minimum instead of zigzagging.

Train / Validation / Test Split

Textbook / Practice Test / Final Exam.

Training set (60-80% of your data): Your textbook. The model studies from this data, adjusting its parameters to learn patterns. It sees these examples many times.

Validation set (10-20%): Your practice tests. After training, you test the model on data it has NOT seen. You use the results to tweak hyperparameters (like "should I use learning rate 0.01 or 0.001?"). You can retrain and recheck validation many times.

Test set (10-20%): The sealed final exam. Touched ONLY ONCE, after all tuning is done. This gives an honest estimate of real-world performance. If you peek at it during tuning, your estimate becomes falsely optimistic.

Stratified split (for imbalanced data): If only 5% of transactions are "fraud," a random split might put all fraud in training and none in test (or vice versa). Stratified splitting ensures each split keeps the same class ratio (5% fraud in train, 5% in validation, 5% in test).

Chapter 3: Linear Regression

What we're trying to do: You have data on house sizes and prices. You plot them: size on the x-axis, price on the y-axis. The dots go upward in a rough line. You want to draw the *single best straight line* through those dots so you can predict the price of any new house by just reading off the line.

The Line (Hypothesis)

$h(x) = \theta_0 + \theta_1 x$

θ_0 (theta-zero) is the **y-intercept**: where the line crosses the y-axis. In the house example, it's the "base price" when size is 0. (Theoretical -- no house has 0 size, but the math needs a starting point.)

θ_1 (theta-one) is the **slope**: how steeply the line goes up. It tells you "for every 1 unit increase in x, the predicted y increases by θ_1."

Example: If θ_0 = 10,000 and θ_1 = 200, then the line is h(x) = 10,000 + 200x. For a 100 m² house: h(100) = 10,000 + 200(100) = 30,000. Each extra m² adds exactly 200 rupees.

Cost Function: "How Bad Is My Current Line?"

There are infinitely many possible lines. We need a way to *score* each line -- to measure how badly it fits the data. Then we can search for the line with the best (lowest) score.

The idea: For each data point, measure the *error* (how far the line's prediction is from the actual value). Then combine all errors into one single number. The line with the smallest combined error wins.

$J(\theta) = (1/2m) \Sigma (h(x_i) - y_i)^2$

Let me explain EVERY piece, one at a time:

$h(x_i) - y_i$: This is the error for one data point. "What did my line predict?" minus "What was the actual value?" If the line predicted 5000 but the actual price was 4500, the error is +500 (overpredicted). If it predicted 3000 but actual was 4000, error is -1000 (underpredicted).

$(...)^2$ (squaring): Why square it? Two reasons. (1) Without squaring, an error of +500 and an error of -500 would add to zero, making us think the line is perfect when it's actually off by 500 in both directions. Squaring makes everything positive. (2) Squaring penalizes large errors much more. An error of 10 contributes 100. An error of 100 contributes 10,000. This forces the algorithm to really care about fixing big mistakes.

Σ (sigma, the sum): Add up the squared errors for ALL data points. If you have 100 houses, you compute 100 errors, square each, and add them all up.

1/m (divide by total number of points): This gives the AVERAGE squared error. Without this, having more data would automatically mean a bigger cost (just because there are more terms to add). Dividing by m normalizes it.

1/2 (the half): A mathematical convenience. When we take the derivative later, the exponent 2 comes down (from the chain rule) and cancels with this 1/2. It makes formulas cleaner. It doesn't change which line is best.

Worked Example: 3 data points: (1,2), (2,4), (3,5). Our current line: $h(x) = 1 + 1.5x$.

Point 1 (x=1, actual y=2): Prediction = 1 + 1.5(1) = 2.5. Error = 2.5 - 2 = +0.5. Squared = 0.25.
Point 2 (x=2, actual y=4): Prediction = 1 + 1.5(2) = 4.0. Error = 4.0 - 4 = 0.0. Squared = 0.00.
Point 3 (x=3, actual y=5): Prediction = 1 + 1.5(3) = 5.5. Error = 5.5 - 5 = +0.5. Squared = 0.25.

Cost $J = (1 / (2 \times 3)) \times (0.25 + 0.00 + 0.25) = (1/6) \times 0.50 =$ **0.083**.

This number (0.083) is our "badness score" for this line. A perfect line through all points would give J = 0. A terrible line might give J = 50. We want to find the θ_0 and θ_1 that make J as small as possible.

Gradient Descent: Automatically Finding the Best Line

The Fog on a Hill. Imagine you are dropped somewhere on a hilly landscape in thick fog. You cannot see the valley below. But you CAN feel the slope under your feet. Your strategy:

1. Feel which direction slopes downhill.
2. Take one small step in that direction.
3. Feel the slope again (it might have changed).
4. Take another step downhill.
5. Repeat until the ground feels flat -- you've found the valley bottom.

The valley bottom is where the cost function J is at its minimum. The location at the bottom tells you the best θ_0 and θ_1.

What is "the slope" in math? It's the *partial derivative* (gradient) of J. It tells you: "if I increase θ by a tiny amount, does J go up or down, and by how much?"

What controls step size? The *learning rate* α (alpha). Big α = big steps (faster but might overshoot). Small α = tiny steps (safer but painfully slow).

The update rule: $\theta\_new = \theta\_old - \alpha \times gradient$

The **minus sign** is crucial: we go in the OPPOSITE direction of the gradient. The gradient points uphill (direction of steepest ascent). We want to go downhill, so we subtract it.

Gradient for θ_0: $(1/m) \Sigma (h(x_i) - y_i)$
In plain words: average all the errors. If most predictions are too high (positive errors), this gradient is positive, so subtracting it decreases θ_0 (moves the line down). If predictions are too low (negative errors), gradient is negative, subtracting it increases θ_0 (moves the line up).

Gradient for θ_1: $(1/m) \Sigma (h(x_i) - y_i) \times x_i$
In plain words: for each error, multiply it by the input x that caused it, then average. Why multiply by x? Because θ_1 is the slope. A data point at x=100 has much more "leverage" on the slope than a point at x=1. Multiplying by x gives more weight to points with larger x values.

FULL Worked Example (every single step).
Data: (1,2), (2,5), (3,7). Start with θ_0=0, θ_1=1, α=0.05.

Step 1: Compute predictions using $h(x) = 0 + 1 \times x = x$.
h(1) = 1, h(2) = 2, h(3) = 3.

Step 2: Compute errors (prediction - actual).
e1 = 1 - 2 = **-1** (predicted 1, actual 2, too low by 1)
e2 = 2 - 5 = **-3** (predicted 2, actual 5, too low by 3)
e3 = 3 - 7 = **-4** (predicted 3, actual 7, too low by 4)
All errors are negative = our line is consistently BELOW the data. Needs to go up and get steeper.

Step 3: Gradient for θ_0 = average of errors = (1/3)(-1 + -3 + -4) = (1/3)(-8) = **-2.667**
Negative gradient means: the line needs to shift UP (increase θ_0).

Step 4: Gradient for θ_1 = average of (error × x)
= (1/3)[(-1)(1) + (-3)(2) + (-4)(3)] = (1/3)[-1 + -6 + -12] = (1/3)(-19) = **-6.333**
Negative gradient means: the slope needs to INCREASE (line needs to be steeper).

Step 5: Update
θ_0 = 0 - 0.05 × (-2.667) = 0 + 0.133 = **0.133** (moved up)
θ_1 = 1 - 0.05 × (-6.333) = 1 + 0.317 = **1.317** (got steeper)

Result: Line went from h = 0 + 1x to h = 0.133 + 1.317x. It shifted up and got steeper -- exactly what the errors told us to do. This is ONE iteration. Repeat hundreds of times until the line barely changes.

With Ridge Regularization (λ=2)

> Ridge regularization adds an extra pull on θ_1 toward zero. The extra term in the gradient is $(\lambda/m) \times \theta_1$. This prevents the slope from becoming too extreme. We never regularize θ_0.

Using same values: gradient for θ_1 = -6.333 + (2/3)(1) = -6.333 + 0.667 = **-5.667**
θ_1 = 1 - 0.05(-5.667) = 1 + 0.283 = **1.283**

Compare: Without Ridge: 1.317. With Ridge: 1.283. Ridge dampened the update slightly, preventing the slope from growing as fast.

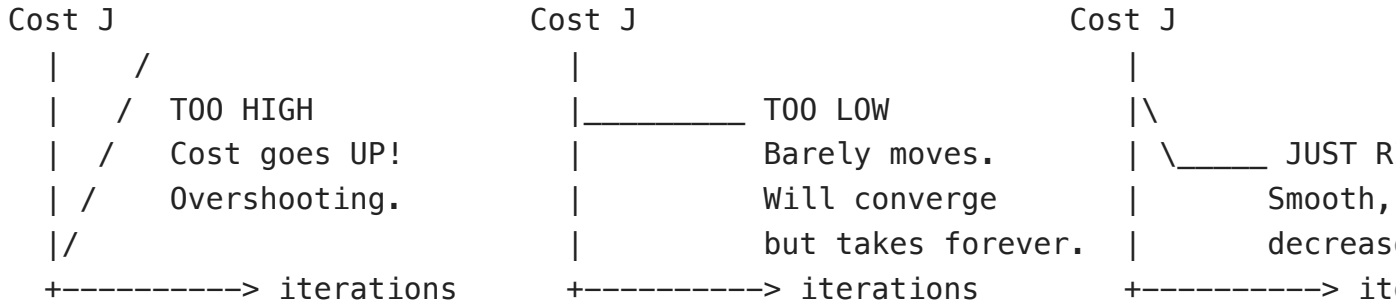
Normal Equation (Skip Gradient Descent Entirely)

$\theta = (X^TX)^{-1}X^Ty$

One matrix computation gives you the exact best θ directly. No iterations, no learning rate, no tuning. But it's slow for large datasets (inverting a big matrix is expensive).

Dimensions: m=28 samples, n=4 features (+1 bias column = 5 cols).
X: 28×5, X^T: 5×28, X^TX: 5×5, $(X^TX)^{-1}$: 5×5, X^Ty: 5×1, **θ: 5×1**.

Learning Rate: What Happens When It's Wrong



Types of Gradient Descent

Type	Data used per step	Analogy	Behavior
Batch GD	ALL m data points	Read the entire textbook before updating notes	Slow per step, smooth convergence
Stochastic GD	1 random point	Read one page, immediately update notes	Very fast per step, noisy/bouncy path
Mini-batch GD	Small batch (e.g., 32)	Read one chapter, then update notes	Best of both: fast and fairly smooth

Chapter 4: Regularization

The Problem: Memorizing vs Understanding.

Imagine a student who memorizes every single question-answer pair from the textbook. Ask them the EXACT same question, they get it right. Change one word, they are completely lost. They never understood the underlying concept -- they just memorized.

This is **overfitting**. The model has learned the training data too perfectly -- including its random noise and quirks. It gets 99% on training data but 60% on new data it has never seen.

Regularization is the fix. It adds a penalty to the cost function that punishes the model for being too complex. It says: "yes, try to fit the data, but don't go overboard. Keep your coefficients reasonable."

Ridge (L2): "Turn Down the Volume"

New Cost = Original MSE + $\lambda \Sigma \theta_i^2$

Analogy: Each coefficient θ is a volume knob on a stereo. Ridge turns ALL knobs down a bit -- making everything quieter (closer to zero) but never fully silencing any knob. Every feature stays in the model, just with a dampened influence.

Lasso (L1): "Eliminate Contestants"

New Cost = Original MSE + $\lambda \Sigma |\theta_i|$

Analogy: A reality show judge who eliminates contestants. Lasso doesn't just turn down the volume -- it can push some coefficients to *exactly zero*. If a feature is not useful enough to justify its presence, Lasso removes it entirely. This is automatic **feature selection**.

Property	Ridge (L2)	Lasso (L1)
Penalty	Sum of squared coefficients ($\Sigma \theta^2$)	Sum of absolute values ($\Sigma \|\theta\|$)
Effect on coefficients	Makes ALL of them smaller, but none become exactly zero	Makes some EXACTLY zero (removes them)
Feature selection?	No -- keeps every feature, just dampened	Yes -- automatically drops useless features
When to use	You think all features probably matter at least a little	You suspect many features are irrelevant noise

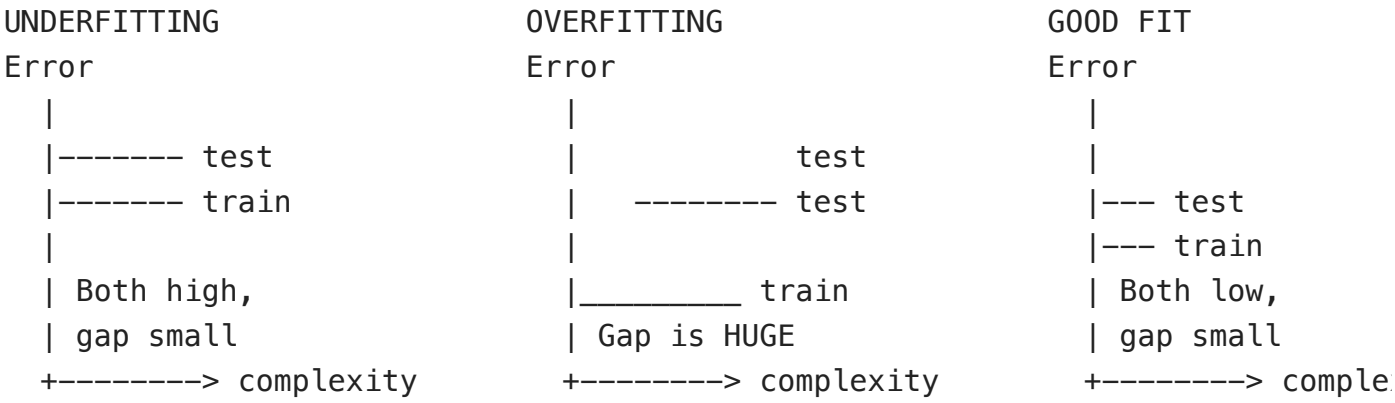
The λ Dog Leash. λ controls how strong the penalty is.

λ too small (loose leash): The dog runs wild. The model has too much freedom and overfits.
λ too large (tight leash): The dog can barely move. All coefficients are squished to near-zero. The model is too simple and underfits.
λ just right: The dog explores but stays close. The model captures real patterns without memorizing noise.

Chapter 5: Bias-Variance Tradeoff

> **Every model's error comes from two sources:**
>
> **Bias (error from oversimplifying):** The model is too simple to capture the real patterns. Imagine trying to fit a straight line through data that clearly curves -- the line will ALWAYS be wrong, no matter how much data you give it. High bias = underfitting.
>
> **Variance (error from over-sensitivity):** The model is so complex that it memorizes the training data, including random noise. If you train it on a slightly different dataset, you get a completely different model. High variance = overfitting.
>
> **The tradeoff:** Making the model more complex reduces bias but increases variance. Making it simpler reduces variance but increases bias. The goal is the sweet spot.

Train Error	Test Error	Gap	Diagnosis	What to do
HIGH	HIGH	Small	**Underfitting** (too simple, high bias)	More features, more complex model, reduce λ
LOW	HIGH	LARGE	**Overfitting** (memorized training, high variance)	More data, regularization, simpler model, increase λ
LOW	LOW	Small	**Good fit!**	You're done. Deploy it.



Practice: Diagnose 4 Models.

Model	Train RMSE	Test RMSE	R^2	Your Diagnosis
A	15.2	15.8	0.45	**Underfitting.** Both errors are high (~15). The gap is tiny (0.6). R^2 = 0.45 means the model explains only 45% of the variation. It is too simple.
B	0.3	14.7	0.92 / 0.48	**Overfitting.** Train error is almost zero (0.3) but test error is terrible (14.7). Gap = 14.4! R^2 drops from 92% to 48%. It memorized the training data.
C	2.1	2.8	0.94	**Good fit.** Both errors are low, gap is small (0.7). R^2 = 94%. Generalizes well to new data.
D	1.0	9.5	0.97 / 0.61	**Overfitting.** Train = 1.0, test = 9.5 -- massive gap. R^2 crashes from 97% to 61%. Needs regularization or more data.

Chapter 6: Logistic Regression

TRAP: Despite the name "regression," logistic regression is used for **CLASSIFICATION** (yes/no, spam/not, sick/healthy). The name comes from the logistic (sigmoid) function it uses inside.

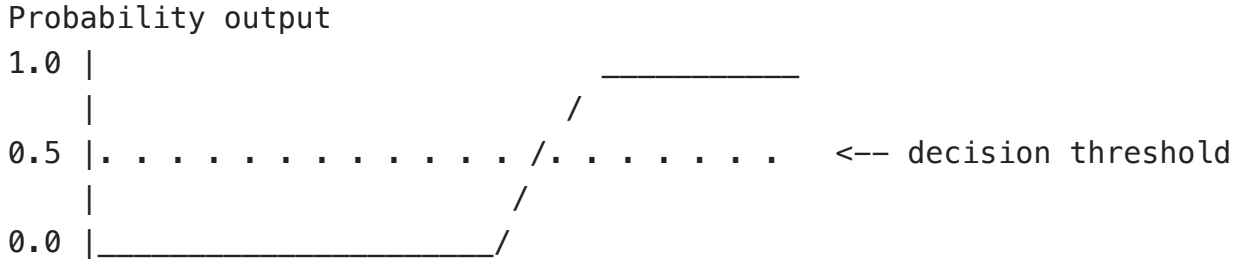
The Problem That Logistic Regression Solves

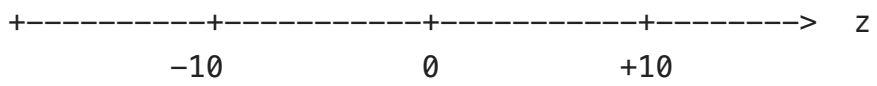
Linear regression outputs any number: -500, 0, 3.7, 1000000. But for classification, we need a probability between 0 and 1 ("there is a 73% chance this email is spam").

How do we convert "any number" into "a number between 0 and 1"? We use the **sigmoid function**. No matter what number you feed into it, the output is always between 0 and 1.

Sigmoid: $\sigma(z) = 1 / (1 + e^{-z})$

What it does to different inputs:
When z is very positive (like +10): e^{-10} is basically 0. So 1/(1+0) = 1. Output near 1 = "very confident this is class 1."
When z is very negative (like -10): e^{10} is huge (~22000). So 1/(1+22000) is basically 0. Output near 0 = "very confident this is class 0."
When z = 0: e^{0} = 1. So 1/(1+1) = 0.5. Output = 0.5 = "totally uncertain, could be either class."





How logistic regression works, start to finish:

Step 1: Compute $z = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \ldots$ (This is just linear regression! A weighted sum of features.)

Step 2: Feed z into the sigmoid: $p = 1/(1+e^{-z})$. Now z (which could be any number) has been converted to p (a probability between 0 and 1).

Step 3: Apply a threshold. If $p \geq 0.5$, predict class 1. If $p < 0.5$, predict class 0.

So logistic regression is really just: linear regression + sigmoid squishing + threshold decision.

Log-Odds: What the Model Actually Learns

$\log(p / (1-p)) = \theta^T x$

What is p/(1-p)? The "odds." If p=0.75 (75% chance of class 1), then odds = 0.75/0.25 = 3. Meaning "3 times more likely to be class 1 than class 0."
Why take the log? Odds range from 0 to infinity. Log-odds range from -infinity to +infinity. This matches the output range of a linear equation, making the math work cleanly.

TRAP: A large θ_2 does NOT mean the outcome will definitely be class 1. It means feature x_2 has a strong influence. But other features and the intercept could push the total z negative. Example: $\theta_2=100$, but $\theta_0=-500$ makes the total z = -400, giving $\sigma(-400)$ near 0 = class 0.

Why Not Use MSE for Logistic Regression?

You might think: "just use the same squared-error cost function as linear regression." But when you combine the sigmoid function with MSE, the resulting cost surface becomes **non-convex** -- it has multiple bumps and valleys. Gradient descent can get stuck in a bump (local minimum) and never find the actual best solution.

Instead, we use **log-loss (cross-entropy)**, which creates a perfectly convex surface (one smooth bowl). Gradient descent always rolls to the global minimum.

Log-Loss: The Right Cost Function for Classification

Log-Loss = -(1/m) Σ [y × log(p) + (1-y) × log(1-p)]

This formula looks scary, but the idea is beautiful. Let me explain it case by case:

CASE 1: The patient actually has cancer (y = 1).
When y=1, the formula simplifies to: cost = -log(p).
Now think about what happens:
- Our model says p = 0.99 (99% chance of cancer -- very confident and CORRECT):
  cost = -log(0.99) = 0.01. Tiny cost! Good job, model.
- Our model says p = 0.50 (50% chance -- uncertain):
  cost = -log(0.50) = 0.69. Medium cost. Not great.
- Our model says p = 0.01 (1% chance -- very confident and WRONG!):
  cost = -log(0.01) = 4.6. ENORMOUS cost! You were confidently wrong. Huge punishment.

CASE 2: The patient is healthy (y = 0).
When y=0, the formula simplifies to: cost = -log(1-p).
- Model says p = 0.01 (1% chance of cancer -- correctly says "healthy"):
  cost = -log(0.99) = 0.01. Tiny. Correct and confident.
- Model says p = 0.99 (99% chance of cancer -- WRONG! says sick but they're healthy):
  cost = -log(0.01) = 4.6. Huge penalty for being confidently wrong.

The pattern: Correct and confident = almost zero cost. Wrong and confident = massive cost. This is exactly what we want: the model is rewarded for being right and severely punished for being confidently wrong.

Decision Boundary

The **decision boundary** is the line (or curve) where the model is exactly 50-50 uncertain. On one side it predicts class 1, on the other side class 0. Mathematically, it is where $\theta^T x = 0$ (because sigmoid(0) = 0.5).

With plain features (x_1, x_2), the boundary is a straight line. Add polynomial features and it can curve!

Example producing an ellipse: Features = [1, x_1, x_2, x_1^2, x_2^2], weights w = [-36, 0, 0, 4, 9].
Set $\theta^T x = 0$: $-36 + 0(x_1) + 0(x_2) + 4x_1^2 + 9x_2^2 = 0$
Simplify: $4x_1^2 + 9x_2^2 = 36$. Divide by 36: $x_1^2/9 + x_2^2/4 = 1$.
This is an **ellipse**! Points inside = one class, points outside = the other.

Sigmoid weights and overfitting: If you multiply all weights by a constant (w×1, w×10, w×100), the sigmoid becomes steeper and steeper. At w×100, it is practically a vertical step function -- the model is 100% confident about EVERY point, even those right on the boundary. This extreme confidence = overfitting. Regularization keeps weights reasonable.

Chapter 7: Confusion Matrix & Metrics

This topic is the easiest 5-6 marks on any ML exam. But the formulas can be confusing if you just memorize them. So let me explain the CONCEPT behind each metric first, and then the formula will make perfect sense.

Setting Up the Example

Scenario: A hospital tests 1000 patients for cancer. In reality, 150 of them actually have cancer and 850 are healthy. Our ML model looks at each patient's data and predicts "cancer" or "healthy."

Now the model is not perfect. It will make some correct predictions and some mistakes. There are exactly 4 possible outcomes for any patient:

True Positive (TP): The patient actually HAS cancer, and our model correctly said "cancer." We caught it. Great!
False Negative (FN): The patient actually HAS cancer, but our model said "healthy." We MISSED the cancer. Dangerous!
False Positive (FP): The patient is actually HEALTHY, but our model said "cancer." A false alarm. Scary for the patient, but at least they're fine.
True Negative (TN): The patient is actually HEALTHY, and our model correctly said "healthy." Correctly cleared.

The Confusion Matrix

	Model said "CANCER" (+)	**Model said "HEALTHY" (-)**	**Total Actually**
Actually HAS cancer	**TP = 120** (correctly caught)	**FN = 30** (missed! dangerous!)	150
Actually HEALTHY	**FP = 85** (false alarm)	**TN = 765** (correctly cleared)	850
Total Predicted	205	795	1000

Reading tip: The rows are REALITY (what the patient actually is). The columns are PREDICTION (what the model said). T means the model was right. F means the model was wrong. P means the model said positive. N means the model said negative.

The 5 Metrics (Each One Explained from Scratch)

1. ACCURACY: "Of all 1000 patients, how many did we get right?"

We got it right when we said "cancer" and they had cancer (TP=120), and when we said "healthy" and they were healthy (TN=765). That's 120 + 765 = 885 correct out of 1000 total.

Formula: Accuracy = (TP + TN) / Total = (120 + 765) / 1000 = 88.5%

Why this is MISLEADING: Imagine a lazy model that ALWAYS says "healthy" for everyone. It never predicts cancer at all. For the 850 healthy patients, it's correct. For the 150 cancer patients, it's wrong. Accuracy = 850/1000 = 85%. Sounds great! But it caught ZERO cancers. Every single cancer patient walked away thinking they're fine. That's why accuracy alone is dangerous for imbalanced data.

2. PRECISION: "Of everyone we FLAGGED as cancer, how many actually had cancer?"

Our model flagged 205 people as having cancer (the "Predicted CANCER" column). Of those 205 people, how many truly had cancer? Only 120. The other 85 were false alarms.

Formula: Precision = TP / (TP + FP) = 120 / (120 + 85) = 120 / 205 = 58.5%

What it tells you: "How trustworthy are my positive predictions?" Only 58.5% of our cancer flags were correct. If you're flagged, there's a 41.5% chance it's a false alarm.

When it matters most: Spam filtering. If your spam filter puts a real email in spam (false positive), you might miss an important message from your boss. You want very few false alarms = high precision.

3. RECALL (Sensitivity): "Of everyone who ACTUALLY has cancer, how many did we catch?"

150 people actually have cancer (the "Actually HAS cancer" row). Of those 150, we correctly identified 120. We missed 30.

Formula: Recall = TP / (TP + FN) = 120 / (120 + 30) = 120 / 150 = 80%

What it tells you: "How thorough am I at catching real positives?" We caught 80% of all cancer cases. But 20% walked away undiagnosed.

When it matters most: Cancer screening. Missing a cancer case (false negative) can be fatal. You want to catch as many real cases as possible = high recall. Even if that means more false alarms.

4. SPECIFICITY: "Of everyone who is ACTUALLY healthy, how many did we correctly clear?"

850 people are actually healthy (the "Actually HEALTHY" row). Of those 850, we correctly said "healthy" for 765. We wrongly alarmed 85.

Formula: Specificity = TN / (TN + FP) = 765 / (765 + 85) = 765 / 850 = 90%

What it tells you: "How good am I at correctly reassuring healthy people?" We correctly cleared 90% of healthy patients.

5. F1 SCORE: A single number that balances Precision and Recall.

Sometimes you need one number that captures both. The F1 score is the *harmonic mean* of precision and recall. Unlike the regular average, the harmonic mean penalizes cases where one is much lower than the other.

Formula: F1 = 2 × (Precision × Recall) / (Precision + Recall)
= 2 × (0.585 × 0.80) / (0.585 + 0.80)
= 2 × 0.468 / 1.385
= 0.936 / 1.385 = **0.676**

If either precision or recall is very low, F1 will also be low. You can't hide a weakness behind a strong score in the other metric.

Cost Analysis: When Mistakes Have Different Price Tags

In real life, not all mistakes are equally bad.

Cost of False Negative (missed cancer) = $50,000 (delayed treatment, potential death, lawsuits).
Cost of False Positive (unnecessary biopsy on a healthy person) = $2,000 (biopsy cost, patient anxiety).

Total cost = (30 FN × $50,000) + (85 FP × $2,000) = $1,500,000 + $170,000 = **$1,670,000**

Since FN is 25x more expensive than FP, we should **lower the decision threshold** from 0.5 to maybe 0.3. This means: if the model gives even a 30% probability of cancer, flag it. We will catch more cancers (reducing expensive FNs) at the cost of more false alarms (increasing cheap FPs). Net total cost goes down.

Chapter 8: Decision Trees

The "20 Questions" Game. One person thinks of something. The other asks yes/no questions to figure it out. "Is it alive?" "Is it bigger than a car?" Each question narrows down the possibilities.

A decision tree does the same thing with data. It asks a series of yes/no questions about the features to classify each data point. The big question is: **which question should you ask FIRST?** The one that gives the most useful information -- the one that best separates the classes.

Entropy: Measuring How "Confused" a Set Is

Concept first (no formula yet). Imagine a bag of marbles:

Bag 1: 10 red marbles, 0 blue. You reach in. What will you get? Red. You're 100% certain. **Zero confusion.**

Bag 2: 5 red, 5 blue. You reach in. What will you get? Could be either. You have **maximum confusion.**

Bag 3: 9 red, 1 blue. You reach in. Probably red, but maybe blue. **Low confusion, but not zero.**

Entropy is a number that measures this confusion. It goes from 0 (no confusion, perfectly pure) to 1 (maximum confusion for 2 classes, 50-50 split).

Formula: $H(S) = -\Sigma\, p_i \log_2(p_i)$

Where p_i is the proportion of class i. Why $\log_2$? We measure information in **bits** (binary digits). One bit is the answer to one yes/no question.

Full Worked Example: Building a Decision Tree Step by Step

Dataset: 14 data points. 9 labeled "Rain" (R) and 5 labeled "Dry" (D). We want to build a tree that predicts Rain vs Dry.

STEP 1: Calculate entropy of the entire (unsplit) dataset.
p(Rain) = 9/14 = 0.643. p(Dry) = 5/14 = 0.357.

$H(S) = -(9/14) \times \log_2(9/14) - (5/14) \times \log_2(5/14)$

Let me compute each piece:
$\log_2(9/14) = \log_2(0.643) = -0.637$ (from the log table: 9/14 is between 1/2 and 3/4)
$\log_2(5/14) = \log_2(0.357) = -1.486$

$H(S) = -(0.643)(-0.637) - (0.357)(-1.486)$
$= 0.410 + 0.531$
$=$ **0.940 bits**

This is close to 1.0 (the maximum), which makes sense -- we have 9 of one class and 5 of the other, which is fairly mixed.

STEP 2: Try splitting on the "Income" feature.
When we split on Income (High vs Low), we get two groups:

High Income group: 8 data points (6 Rain, 2 Dry).
$H(\text{High}) = -(6/8)\log_2(6/8) - (2/8)\log_2(2/8)$
$= -(0.75)(-0.415) - (0.25)(-2.0)$
$=$ 0.311 + 0.500 = **0.811 bits**

Low Income group: 6 data points (3 Rain, 3 Dry).
$H(\text{Low}) = -(3/6)\log_2(3/6) - (3/6)\log_2(3/6)$
$= -(0.5)(-1) - (0.5)(-1)$
$=$ 0.5 + 0.5 = **1.000 bits** (50-50 = maximum confusion!)

STEP 3: Weighted average entropy AFTER the split.
The High group has 8/14 of the data, and the Low group has 6/14. We weight each group's entropy by its size:
$H(S|\text{Income}) = (8/14) \times 0.811 + (6/14) \times 1.000$
$=$ 0.463 + 0.429 = **0.892 bits**

STEP 4: Information Gain (the payoff).
IG = entropy before - entropy after = 0.940 - 0.892 = **0.048 bits**

What does this mean? Splitting on Income reduced our confusion by only 0.048 bits. Not great -- the Low Income group is still 50-50 confused! We would now compute IG for every other feature (Wind, Humidity, Temperature, etc.) and pick the one with the **highest IG** as the root node. That is the **ID3 algorithm**.

Information Content

$I(\text{event}) = -\log_2(p)$

"How surprised are you when this event happens?" Rare events are more surprising and carry more information.

Rain in a desert (p=0.01): $I = -\log_2(0.01) = 6.64$ bits. Very surprising!
Sunshine in a desert (p=0.99): $I = -\log_2(0.99) = 0.014$ bits. Not surprised at all.

Minimum bits to encode n outcomes: $\lceil \log_2(n) \rceil$
5 outcomes: $\log_2(5) = 2.32$. Ceiling: **3 bits** (3 yes/no questions can distinguish $2^3=8$ things, which covers 5).

Continuous Attributes

If a feature is continuous (temperature: 18.5, 22.3, 25.1, 30.0, ...), you can't split on categories. Instead: sort all values, compute the midpoint between each pair of consecutive values, evaluate IG for each possible split point ("is temp > 23.7?"), and pick the midpoint with the highest IG.

Overfitting in Decision Trees

Trees are very prone to overfitting. Without limits, a tree can keep splitting until every leaf has one data point -- 100% training accuracy but useless on new data.

Pre-pruning: Stop growing early. "Stop if IG < threshold" or "stop if node has < 5 samples" or "stop at max depth."

Post-pruning: Grow the full (overfitted) tree first. Then walk back up and remove branches that don't improve validation accuracy. More reliable because you see the full picture before cutting.

MDL (Minimum Description Length): Best tree minimizes [bits to describe the tree] + [bits to describe its errors].

Useful $\log_2$ Values (memorize for the exam)

x	1/8	1/4	1/3	1/2	2/3	3/4	1	2	3	4	5	8
$\log_2(x)$	-3	-2	-1.585	-1	-0.585	-0.415	0	1	1.585	2	2.322	3

Chapter 9: Quick Topics

Linear Basis Functions

Instead of a straight line $y = w_0 + w_1x$, transform x using functions: $y = w_0 + w_1\varphi_1(x) + w_2\varphi_2(x)$. For example, $\varphi_1(x)=x$ and $\varphi_2(x)=x^2$ gives a parabola. The model is still "linear in the weights" (we solve for w's the same way as linear regression), even though the curve through the data can be non-linear.

Discriminant Functions & Multi-class

One-vs-All (OvA): For K classes, train K binary classifiers. Each one asks "is it class i or not?" Pick the class whose classifier is most confident. Need K classifiers.
One-vs-One (OvO): Train a classifier for every pair. K(K-1)/2 classifiers. Each votes. Most votes wins. OvA is more common (fewer classifiers).

Decision Theory & Reject Option

Assign x to the class with the highest P(class|x). **Reject option:** If the highest probability is below a threshold (e.g., < 0.7), refuse to classify and defer to a human expert. Useful in medicine/law where a wrong answer is worse than "I don't know."

Minimum Description Length (MDL)

> The best model minimizes: [bits to describe the model] + [bits to describe errors the model makes]. A complex model has zero errors but costs many bits to describe itself. A simple model costs few bits but has many errors. MDL finds the balance -- Occam's Razor formalized mathematically.

Missing Attributes in Decision Trees

> If a test sample is missing the feature a node asks about: (1) Use the most common value at that node, (2) Send it down all branches weighted by training proportions (fractional instances), or (3) Use a surrogate split on a correlated feature.

Chapter 10: Model Robustness

Model	Effect of Noise	Effect of Outliers
Linear Regression	Moderate. Noise shifts the line slightly but averages out with enough data.	**Very sensitive.** MSE squares errors, so one outlier with error=100 contributes 10,000 to the cost, dragging the entire line toward it.
Logistic Regression	Moderate. Can flip predictions near the boundary but points far from it are stable (sigmoid saturates).	Less sensitive than LR. Sigmoid squashes extremes. But outliers near the boundary can shift it. Regularization helps.
Decision Trees	**Very sensitive.** Noise creates spurious splits. Trees overfit noisy data easily.	Fairly robust. An extreme value just ends up in its own leaf without affecting other splits. Pruning fixes noise issues.

Chapter 11: Decision Table -- "If the Exam Says X, Use Y"

Exam Says...	Use This	Key Formula / Detail
Predict a continuous number	Linear Regression	$h(x)=\theta^T x$, $J=(1/2m)\Sigma(h-y)^2$
Classify into 2 classes	Logistic Regression	$\sigma(z)=1/(1+e^{-z})$, use log-loss
"Which feature to split on?"	Information Gain	IG = H(parent) - weighted H(children)
"Calculate entropy"	Entropy formula	$H = -\Sigma\ p\ \log_2(p)$
Overfitting / high variance	Regularization / more data	Ridge: $+\lambda\Sigma\theta^2$, Lasso: $+\lambda\Sigma\lvert\theta\rvert$
Underfitting / high bias	More features / complex model	Add polynomial features, reduce λ
"Which feature to remove?"	Lasso (L1)	Drives coefficients to exactly 0
"All features matter, reduce complexity"	Ridge (L2)	Shrinks all coefficients, keeps all non-zero
Confusion matrix metrics	TP/FP/TN/FN table	Prec=TP/(TP+FP), Rec=TP/(TP+FN)
"Cost of misclassification"	Adjust threshold	High FN cost = lower threshold

"Closed-form solution"	Normal Equation	$\theta=(X^TX)^{-1}X^Ty$
"Lazy learner"	kNN	No training; finds nearest neighbors at test time
"Generative model"	Naive Bayes	Models P(x|class), uses Bayes' theorem
"Discriminative model"	Logistic Regression / SVM	Directly models boundary or P(class|x)
"Non-linear decision boundary"	Polynomial/basis features	x^2 terms create curves/ellipses
"Scale features"	Z-score or Min-Max	Z=(x-mean)/SD, MinMax=(x-min)/(max-min)
"Batch vs SGD"	Know 3 types	Batch=all, SGD=1, Mini-batch=subset
"Prune the tree"	Pre or Post pruning	Pre=stop early, Post=grow full then trim
"How many bits for N things?"	Ceiling of $\log_2$	$\lceil \log_2(N) \rceil$

Continue to ML_Practice.html for all past paper questions and a mock exam.